You can save this as:

`Java_Level_3_OOP_Core_Java.md`

Java Level 3 — OOP & Core Java

Overview

Level 3 focuses on **Object-Oriented Programming and Core Java concepts**.

By the end of this level, you should be comfortable with:

- Classes & Objects
- Constructors
- `this`
- `static`
- `final`
- Encapsulation
- Getters / Setters
- Inheritance
- `super`
- Method Overloading
- Method Overriding
- Polymorphism
- Abstract Classes
- Interfaces
- Composition vs Inheritance
- Enums
- Records
- Sealed Classes

---

1. Classes & Objects

A **class** is a blueprint for creating objects.

An **object** is an instance of a class.

Example

```
class Employee {

    String name;
    int age;
    double salary;

    void displayDetails() {
        System.out.println(name);
        System.out.println(age);
        System.out.println(salary);
    }
}
```

Creating an object:

```
Employee employee = new Employee();

employee.name = "Ramu";
employee.age = 25;
employee.salary = 100000;

employee.displayDetails();
```

Mental Model

```
Class
  ↓
Blueprint

Object
  ↓
Actual instance created from the blueprint
```

---

2. Constructors

A constructor is used to initialize an object.

It has:

- Same name as the class
- No return type

No-argument constructor

```
class Employee {

    Employee() {
        System.out.println("Employee created");
    }
}
```

Usage:

```
Employee employee = new Employee();
```

Parameterized constructor

```
class Employee {

    String name;
    int age;

    Employee(String name, int age) {
        this.name = name;
        this.age = age;
    }
}
```

Usage:

```
Employee employee =
        new Employee("Ramu", 25);
```

---

3. `this`

`this` refers to the **current object**.

Most commonly, it's used when constructor parameters have the same names as fields.

```
class Employee {

    String name;
    int age;

    Employee(String name, int age) {
```

```
        this.name = name;
        this.age = age;
    }
}
```

Here:

```
this.name
```

means the object's field.

While:

```
name
```

is the constructor parameter.

Constructor chaining with `this()`

You can call another constructor using:

```
this(...)
```

Example:

```
class Employee {

    String name;
    int age;

    Employee() {
        this("Unknown", 0);
    }

    Employee(String name, int age) {
        this.name = name;
        this.age = age;
    }
}
```

Important:

```
this(...)
```

must be the **first statement** in the constructor.

---

4. `static`

`static` belongs to the **class**, rather than individual objects.

Example:

```
class Employee {

    static String companyName = "Yubi";
    static int employeeCount = 0;

    String name;

    Employee(String name) {
        this.name = name;
        employeeCount++;
    }

    static void displayEmployeeCount() {
        System.out.println(
                "Total employees: " + employeeCount
        );
    }
}
```

Usage:

```
Employee employee1 = new Employee("Ramu");
Employee employee2 = new Employee("John");

Employee.displayEmployeeCount();
```

Output:

```
Total employees: 2
```

Important

Access static members using the class:

```
Employee.displayEmployeeCount();
```

rather than:

```
employee1.displayEmployeeCount();
```

---

5. `final`

`final` means something **cannot be changed further**, depending on where it is used.

Final variable

```
final int employeeId = 101;
```

Cannot be reassigned:

```
employeeId = 102; // ❌
```

Final method

```
class Employee {

    final void displayCompanyPolicy() {
        System.out.println("Company policy");
    }
}
```

A child class cannot override it.

Final class

```
final class Employee {
}
```

A final class cannot be extended.

```
class Developer extends Employee { // ❌
}
```

6. Encapsulation

Encapsulation means:

> **Protect an object's internal state and control how it is accessed or modified.**

Instead of:

```
class Employee {

    double salary;
}
```

where anyone can do:

```
employee.salary = -50000;
```

make the field private:

```
class Employee {

    private double salary;
}
```

Then control access through methods.

7. Getters and Setters

Getter

Used to read a value:

```
public double getSalary() {
    return salary;
}
```

Setter

Used to modify a value:

```
public void setSalary(double salary) {

    if (salary < 0) {
        throw new IllegalArgumentException(
                "Salary cannot be negative"
        );
    }

    this.salary = salary;
}
```

Usage:

```
employee.setSalary(100000);

System.out.println(
        employee.getSalary()
);
```

Important

Not every field needs both a getter and setter.

For example:

```
private final int employeeId;
```

might have:

```
public int getEmployeeId() {
    return employeeId;
}
```

but **no setter**.

---

8. Inheritance

Inheritance allows a child class to reuse properties and behavior from a parent class.

The keyword is:

```
extends
```

Example:

```
class Employee {

    String name;
    double salary;
    String department;

    void displayDetails() {
        System.out.println("Name: " + name);
        System.out.println("Salary: " + salary);
        System.out.println("Department: " + department);
    }
}
```

Child:

```
class Developer extends Employee {

    String programmingLanguage;

    void writeCode() {
        System.out.println(
                name + " is writing "
                        + programmingLanguage
                        + " code."
        );
    }
}
```

Usage:

```
Developer developer = new Developer();

developer.name = "Ramu";
developer.salary = 100000;
developer.department = "Engineering";
developer.programmingLanguage = "Java";

developer.displayDetails();
developer.writeCode();
```

Relationship

```
Developer IS-A Employee
```

---

9. `super`

`super` refers to the **parent class**.

There are three important uses:

```
super.field
super.method()
super(...)
```

The most important one for now is `super(...)`.

Calling the parent constructor

```
class Employee {

    String name;
    double salary;
    String department;

    Employee(
            String name,
            double salary,
            String department
    ) {
        this.name = name;
        this.salary = salary;
        this.department = department;
    }
}
```

Child:

```
class Developer extends Employee {

    String programmingLanguage;

    Developer(
            String name,
            double salary,
            String department,
```

```
            String programmingLanguage
    ) {

        super(name, salary, department);

        this.programmingLanguage =
                programmingLanguage;
    }
}
```

The flow is:

```
new Developer(...)
        ↓
Developer constructor
        ↓
super(...)
        ↓
Employee constructor
        ↓
Developer initialization
```

Important:

```
super(...)
```

must be the **first statement** in the constructor.

---

10. Method Overloading

Overloading means:

> **Same method name, different parameters.**

Example:

```
class Calculator {

    int add(int a, int b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }
```

```
    double add(double a, double b) {
        return a + b;
    }
}
```

Usage:

```
Calculator calculator = new Calculator();

calculator.add(10, 20);

calculator.add(10, 20, 30);

calculator.add(10.5, 20.5);
```

Java determines which method to call based on the arguments.

Important

Changing only the return type is **not** overloading.

```
int add(int a, int b) {
}

double add(int a, int b) { // ❌
}
```

The parameters must differ.

---

11. Method Overriding

Overriding occurs when a child class provides its own implementation of a parent method.

Parent:

```
class Employee {

    String name;

    Employee(String name) {
        this.name = name;
    }

    void work() {
        System.out.println(
```

```
        System.out.println(
                name + " is working."
        );
    }
}
```

Child:

```
class Developer extends Employee {

    Developer(String name) {
        super(name);
    }

    @Override
    void work() {
        System.out.println(
                name + " is writing code."
        );
    }
}
```

Usage:

```
Developer developer =
        new Developer("Ramu");

developer.work();
```

Output:

```
Ramu is writing code.
```

Why `@Override`?

It tells Java that you intend to override a parent method.

```
@Override
void work() {
}
```

If you accidentally write:

```
@Override
```

```
void works() {
}
```

Java will detect the error.

---

12. Polymorphism

Polymorphism means:

> **One reference/type can represent different implementations.**

Example:

```
Employee employee1 =
        new Developer("Ramu");

Employee employee2 =
        new Manager("John");
```

Both references are:

```
Employee
```

but the actual objects are different.

```
employee1.work();
employee2.work();
```

Output:

```
Ramu is writing code.
John is managing the team.
```

This is **runtime polymorphism**.

Mental Model

```
Employee
   |
   ├── Developer → work() → Write code
```

```
    │
    └── Manager   → work() → Manage team
```

---

13. Abstract Classes

An abstract class is a base class that is **not meant to be instantiated directly**.

Example:

```
abstract class Employee {

    String name;

    Employee(String name) {
        this.name = name;
    }

    abstract void work();

    void displayName() {
        System.out.println(
                "Employee: " + name
        );
    }
}
```

The abstract method:

```
abstract void work();
```

has no implementation.

Child classes provide it:

```
class Developer extends Employee {

    Developer(String name) {
        super(name);
    }

    @Override
    void work() {
        System.out.println(
                name + " is writing code."
        );
```

```
    }
}
```

You cannot do:

```
Employee employee =
        new Employee("Ramu"); // ❌
```

But you can:

```
Employee employee =
        new Developer("Ramu");
```

Abstract classes can contain

```
Fields
Constructors
Normal methods
Abstract methods
```

---

14. Interfaces

An interface defines a **contract/capability**.

Example:

```
interface Payable {

    void calculateSalary();
}
```

A class implements it:

```
class Developer implements Payable {

    @Override
    public void calculateSalary() {
        System.out.println(
                "Calculating developer salary..."
        );
```

```
    }
}
```

The keyword is:

```
implements
```

Interface polymorphism

```
Payable payable =
        new Developer();

payable.calculateSalary();
```

The reference is:

```
Payable
```

but the actual object is:

```
Developer
```

---

15. Multiple Interfaces

A class can implement multiple interfaces.

```
interface Payable {

    void calculateSalary();
}

interface Reportable {

    void generateReport();
}
```

A class can implement both:

```
class Developer
```

```
        implements Payable, Reportable {

    @Override
    public void calculateSalary() {
        System.out.println(
                "Calculating salary..."
        );
    }

    @Override
    public void generateReport() {
        System.out.println(
                "Generating report..."
        );
    }
}
```

Java does **not** support multiple class inheritance:

```
class Developer
        extends Employee, Person { // ❌
}
```

But multiple interfaces are allowed:

```
class Developer
        implements Payable, Reportable {
}
```

Default methods

Interfaces can contain `default` methods:

```
interface Payable {

    void calculateSalary();

    default void paymentStatus() {
        System.out.println(
                "Payment processing..."
        );
    }
}
```

The implementing class automatically gets the default implementation.

Static interface methods

```
interface Payable {

    static void printPolicy() {
        System.out.println(
                "Payment policy"
        );
    }
}
```

Call it through the interface:

```
Payable.printPolicy();
```

16. Composition vs Inheritance

The easiest way to remember:

```
Inheritance → IS-A
Composition → HAS-A
```

Inheritance

```
class Developer extends Employee {
}
```

Relationship:

```
Developer IS-A Employee
```

Composition

A developer has a laptop:

```
class Laptop {

    String brand;

    Laptop(String brand) {
        this.brand = brand;
    }

    void start() {
        System.out.println(
                brand + " laptop started."
        );
    }
}
```

Developer:

```
class Developer {

    String name;
    Laptop laptop;

    Developer(
            String name,
            Laptop laptop
    ) {
        this.name = name;
        this.laptop = laptop;
    }

    void code() {
        laptop.start();

        System.out.println(
                name + " is coding."
        );
    }
}
```

Usage:

```
Laptop laptop =
        new Laptop("MacBook");

Developer developer =
        new Developer("Ramu", laptop);

developer.code();
```

Relationship

Relationship:

```
Developer HAS-A Laptop
```

Favor composition over inheritance

This doesn't mean never use inheritance.

Use inheritance when there is a genuine:

```
IS-A
```

relationship.

Use composition when there is:

```
HAS-A
```

relationship.

Composition is especially common in backend services:

```
OrderService
    │
    ├── HAS-A → PaymentService
    ├── HAS-A → OrderRepository
    └── HAS-A → NotificationService
```

This concept will become very important with **Spring Boot Dependency Injection**.

---

17. Enums

An enum represents a **fixed set of possible values**.

Example:

```
enum OrderStatus {

    PENDING,
    CONFIRMED,
    SHIPPED,
```

```
    DELIVERED,
    CANCELLED
}
```

Usage:

```
OrderStatus status =
        OrderStatus.PENDING;
```

This is safer than:

```
String status = "PENDNG";
```

because Java validates enum values.

Enum with switch

```
switch (status) {

    case PENDING:
        System.out.println(
                "Waiting for confirmation"
        );
        break;

    case CONFIRMED:
        System.out.println(
                "Order confirmed"
        );
        break;

    case SHIPPED:
        System.out.println(
                "Order is on the way"
        );
        break;

    case DELIVERED:
        System.out.println(
                "Order delivered"
        );
        break;

    case CANCELLED:
        System.out.println(
                "Order cancelled"
```

```
        );
        break;
}
```

Enum with fields and methods

```
enum OrderStatus {

    PENDING("Waiting for payment"),
    CONFIRMED("Order confirmed"),
    SHIPPED("Order shipped"),
    DELIVERED("Order delivered");

    private final String description;

    OrderStatus(String description) {
        this.description = description;
    }

    public String getDescription() {
        return description;
    }
}
```

Usage:

```
System.out.println(
        OrderStatus.SHIPPED.getDescription()
);
```

Output:

```
Order shipped
```

`values()`

Returns all enum values:

```
for (OrderStatus status :
        OrderStatus.values()) {

    System.out.println(status);
}
```

```
}
```

`valueOf()`

Converts a string to an enum:

```
OrderStatus status =
        OrderStatus.valueOf("SHIPPED");
```

Enum matching is case-sensitive.

---

18. Records

Records were introduced in **Java 16**.

They're designed primarily for **data-carrying objects**.

Traditional DTO:

```
class UserDTO {

    private final String name;
    private final String email;
    private final int age;

    UserDTO(
            String name,
            String email,
            int age
    ) {
        this.name = name;
        this.email = email;
        this.age = age;
    }

    public String getName() {
        return name;
    }

    public String getEmail() {
        return email;
    }

    public int getAge() {
        return age;
    }
```

```
}
```

Record:

```
record UserDTO(
        String name,
        String email,
        int age
) {
}
```

Java automatically provides:

- Constructor
- Accessor methods
- `equals()`
- `hashCode()`
- `toString()`

Record accessors

Normal class:

```
user.getName();
```

Record:

```
user.name();
```

Example:

```
UserDTO user =
        new UserDTO(
                "Ramu",
                "ramu@example.com",
                25
        );

System.out.println(user.name());
System.out.println(user.email());
System.out.println(user.age());
```

Records are immutable

You cannot do:

```
user.name = "John"; // ❌
```

Record components are final.

Records can have methods

```
record User(
        String name,
        String email
) {

    public String displayName() {
        return name + " <" + email + ">";
    }
}
```

Records can validate data

```
record User(
        String name,
        String email,
        int age
) {

    public User {

        if (age < 0) {
            throw new IllegalArgumentException(
                    "Age cannot be negative"
            );
        }
    }
}
```

This is called a **compact constructor**.

Backend use case

Records are excellent for DTOs:

```
record CreateUserRequest(
        String name,
        String email,
        String password
) {
}
```

```
record UserResponse(
        long id,
        String name,
        String email
) {
}
```

You'll see this frequently in modern Java/Spring Boot applications.

---

19. Sealed Classes

A sealed class allows you to **control which classes can extend it**.

Normal inheritance:

```
class Employee {
}

class Developer extends Employee {
}

class Manager extends Employee {
}

class Intern extends Employee {
}
```

Potentially any class can extend `Employee`.

Sealed class

```
sealed class Employee
        permits Developer, Manager {
}
```

Now only:

```
Developer
Manager
```

can extend `Employee`.

Example:

```
final class Developer
        extends Employee {
}

final class Manager
        extends Employee {
}
```

But:

```
class Intern extends Employee {
}
```

is invalid.

Permitted subclasses

A permitted subclass must choose one of:

`final`

```
final class Developer
        extends Employee {
}
```

Cannot be extended further.

`sealed`

```
sealed class Developer
        extends Employee
        permits BackendDeveloper,
                FrontendDeveloper {
}
```

Further restricts inheritance.

`non-sealed`

```
non-sealed class Developer
        extends Employee {
}
```

Reopens inheritance.

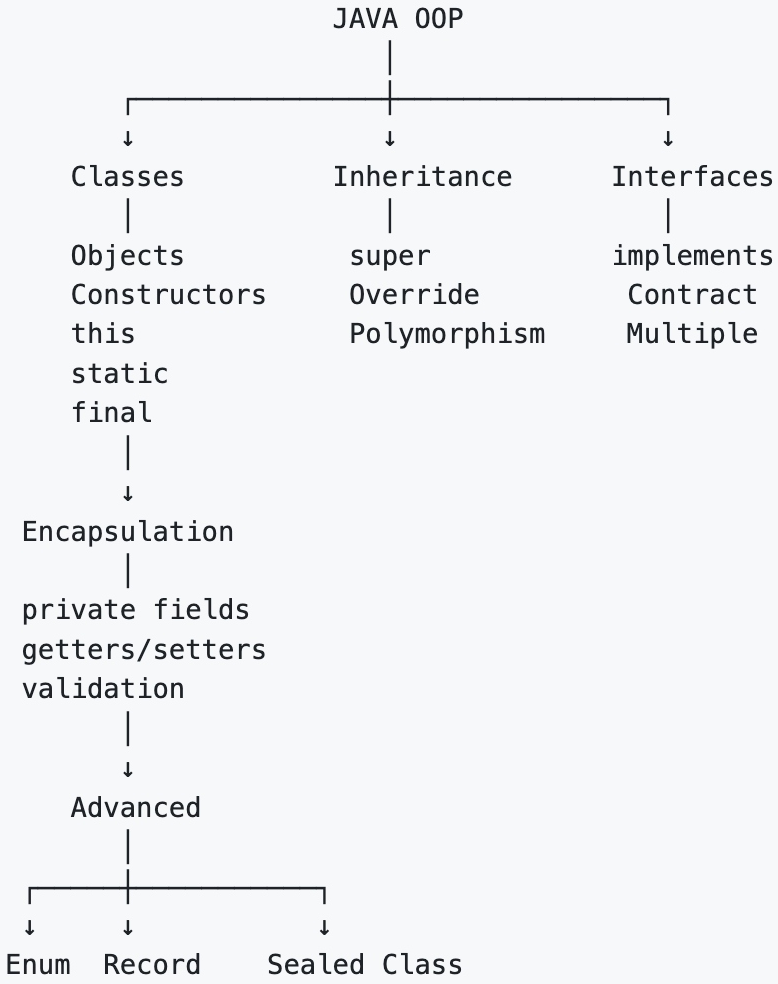
---

20. OOP Concepts — Quick Comparison

Concept	Meaning
Class	Blueprint
Object	Instance of a class
Constructor	Initializes an object
`this`	Current object
`static`	Belongs to class
`final`	Cannot be changed/extended/overridden
Encapsulation	Protect internal state
Inheritance	Reuse through IS-A relationship
`super`	Parent class reference
Overloading	Same method, different parameters
Overriding	Child replaces parent behavior
Polymorphism	Same reference, different behavior
Abstract class	Base class with incomplete/shared behavior
Interface	Contract/capability

Composition	HAS-A relationship
Enum	Fixed set of values
Record	Compact immutable data carrier
Sealed class	Restricted inheritance

🎯 Level 3 — Final Cheat Sheet



The most important interview distinctions

```
Overloading
→ Same class
→ Same method name
→ Different parameters
→ Compile time

Overriding
→ Parent + child
→ Same method signature
```

→ Different implementation
→ Runtime

Inheritance
→ IS-A

Composition
→ HAS-A

Abstract Class
→ Shared base + partial implementation

Interface
→ Contract/capability

Record
→ Compact immutable data carrier

Enum
→ Fixed set of values

Sealed Class
→ Restricted inheritance